AKDENİZ UNİVERSİTY • COMPUTER ENGİNEERİNG
Advanced Web Programming • 2026

FinTreX

CrewAI Implementation Report

Homework 3 — AI Agent Integration & Implementation

Student	Egemen ÇAMÖZÜ (20220808076)
Submitted to	Melih Günay
Document Rev.	v1.0 — June 2026

"Bridging individual investors with professional financial consultants through a data-driven, AI-augmented consultation ecosystem."

Contents

1 Introduction **2**
1.1 Context and HW2 Recap . . . 2
1.2 Project Repository and Structure . . . 2

2 System Architecture **3**

3 CrewAI Implementation **3**
3.1 Agents (`agents.py`) . . . 3
3.2 Tasks (`tasks.py`) . . . 4
3.3 Crew Kickoff (`crew.py`) . . . 5
3.4 FastAPI Entry Point (`main.py`) . . . 5
3.5 Configuration and Dependencies . . . 6

4 .NET Backend Integration **6**
4.1 Service Interface and Implementation . . . 6
4.2 DI Registration and Task Trigger . . . 7

5 Database, Challenges and Security **8**
5.1 Database Schema . . . 8
5.2 Technical Challenges . . . 8
5.3 Security: Dual-Layer Safety . . . 9

6 Working Application Screenshots **9**

7 End-to-End Flow and Conclusion **11**
7.1 Data Flow Summary . . . 11
7.2 Future Enhancements . . . 11
7.3 Conclusion . . . 11

1 Introduction

1.1 Context and HW2 Recap

This report is a **direct continuation of Homework 2**, in which the AI Agent Planning Document for FinTreX was submitted. In HW2, the **Pre-Analysis Assistant (PAA)** was selected as the primary AI agent, its architecture was designed, and its data-flow was documented at a conceptual level. **Homework 3** covers the actual **implementation** using the **CrewAI** framework.

Table 1. HW2 Plan vs. HW3 Implementation

Aspect	HW2 — Plan	HW3 — Built
Agent Framework	CrewAI (proposed)	CrewAI v0.80+ (running)
Communicatio	FastAPI microservice	FastAPI on port 8500
Agent Count	2 agents (proposed)	Data Analyst + Report Writer
.NET Integration	HTTP client (proposed)	`PreAnalysisService` HttpClient
Database	`PreAnalysisReport` (existed)	Used as-is; zero migrations
Safety Rules	No-advice prompts (planned)	Dual-layer: prompt + regex

1.2 Project Repository and Structure

Git Repository

URL: https://github.com/egemencamozu/FinTreX

Project Structure

```
FinTreX/
|-- ai-service/            # Python CrewAI microservice (NEW)
|   |-- main.py
|   |-- crew.py
|   |-- agents.py
|   |-- tasks.py
|   |-- models.py
|   |-- config.py
|   |-- requirements.txt
|-- backend/               # .NET 8 Clean Architecture
|-- frontend/              # Angular 20
+-- mobile/                # Flutter
```

2 System Architecture

The PAA is deployed as a **sidecar Python microservice** communicating with the .NET 8 backend over HTTP:

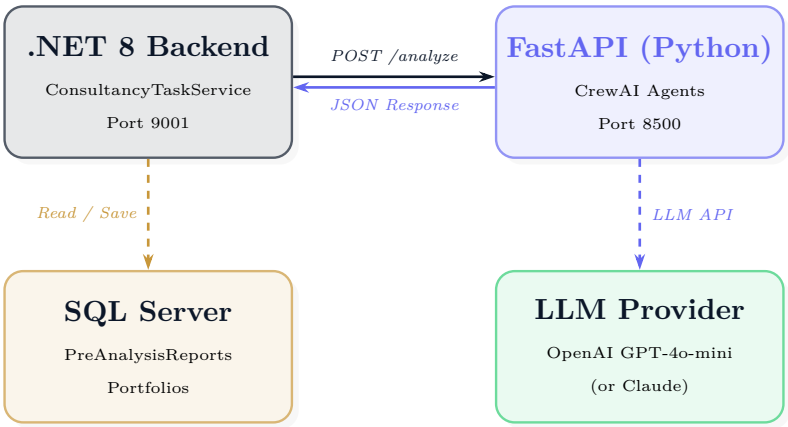

Figure 1. PAA Microservice Communication Architecture

3 CrewAI Implementation

3.1 Agents (`agents.py`)

The crew has **two specialized agents** with strict no-advice constraints:

Table 2. PAA Agent Specifications

Property	Agent 1: Data Analyst	Agent 2: Report Writer
Purpose	Quantitative extraction	Human-readable report
Output	Structured JSON metrics	PreAnalysisReport JSON
Language	English (data)	Turkish (report)
Advice?	Never	Never

```python
from crewai import Agent
from config import get_llm

def create_data_analyst_agent() -> Agent:
    return Agent(
        role="Portfolio Data Analyst",
        goal="Analyze portfolio data and extract objective, "
             "quantitative insights. Calculate asset distribution, "
             "concentration ratios, and type breakdown.",
        backstory="You are a meticulous financial data analyst. "
                  "You NEVER give investment advice. You only "
                  "produce factual, numerical summaries. You "
                  "identify patterns like concentration risk "
                  "purely based on mathematical ratios.",
        verbose=True, allow_delegation=False, llm=get_llm(),
    )

```

```python
def create_report_writer_agent() -> Agent:
    return Agent(
        role="Objective Report Writer",
        goal="Transform quantitative analysis into a clear, "
             "OBJECTIVE report. NO investment advice, NO "
             "buy/sell/hold recommendations.",
        backstory="You are a professional report writer for "
                  "financial data. You present FACTS only. "
                  "Never use 'recommend', 'should', 'consider "
                  "buying'. Use 'observation', 'finding'. "
                  "Write in Turkish.",
        verbose=True, allow_delegation=False, llm=get_llm(),
    )
```

Listing 1. agents.py — Agent definitions with safety constraints

Safety: No-Financial-Advice Policy

Both agents are **explicitly prohibited** from generating investment recommendations. Key constraints embedded in the agent backstories:

- "NEVER give investment advice" — hard-coded in goal/backstory
- Reports use *observation* and *finding* instead of *recommendation*
- Risk level is calculated **only** from diversification and concentration ratios

3.2 Tasks (tasks.py)

```python
from crewai import Task
from agents import (create_data_analyst_agent,
                    create_report_writer_agent)

def create_analysis_task(portfolio_data, task_info):
    data_analyst = create_data_analyst_agent()
    report_writer = create_report_writer_agent()

    analyze_task = Task(
        description=f"""Analyze portfolio data quantitatively.
TASK CONTEXT: {task_info}
PORTFOLIO DATA (JSON): {portfolio_data}
Output MUST include: asset type distribution, portfolio
value per currency, top 3 assets by weight, diversification
score, notable observations. Output as JSON.
DO NOT provide investment advice.""",
        expected_output="JSON with quantitative metrics.",
        agent=data_analyst,
    )

    report_task = Task(
        description=f"""Write an objective pre-analysis report
in Turkish for a licensed economist.
STRICT RULES:
- ASLA yatirim tavsiyesi verme
- ASLA al/sat/tut onerisi yapma
- Sadece VERIYE DAYALI gozlemler yaz
Output MUST be JSON with: "summary", "risk_level"
(Low|Medium|High), "market_outlook", "key_findings"."""
        expected_output="JSON with summary, risk_level, "
                        "market_outlook, key_findings.",
        agent=report_writer,
    )
    return [analyze_task, report_task]
```

Listing 2. tasks.py — Task definitions

3.3 Crew Kickoff (crew.py)

The `Crew` orchestrates both agents in a **sequential process** — Agent 1 completes before Agent 2 starts:

```python
import json
from crewai import Crew, Process
from tasks import create_analysis_task
from models import AnalyzeRequest

def run_pre_analysis_crew(request: AnalyzeRequest) -> dict:
    portfolio_data = json.dumps(
        [p.model_dump() for p in request.user_portfolios],
        ensure_ascii=False, indent=2,
    )
    task_info = (f"Title: {request.task_title}\n"
                 f"Description: {request.task_description}\n"
                 f"Category: {request.task_category}")

    tasks = create_analysis_task(portfolio_data, task_info)

    crew = Crew(
        agents=[t.agent for t in tasks],
        tasks=tasks,
        process=Process.sequential,  # Agent 1 -> Agent 2
        verbose=True,
    )
    return crew.kickoff()
```

Listing 3. crew.py — Crew definition and kickoff

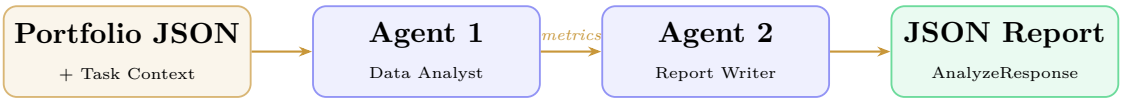

Figure 2. Sequential CrewAI Pipeline

3.4 FastAPI Entry Point (main.py)

```python
import json, traceback
from datetime import datetime, timezone
from fastapi import FastAPI
from models import AnalyzeRequest, AnalyzeResponse
from crew import run_pre_analysis_crew
from config import FASTAPI_PORT

app = FastAPI(title="FinTreX PAA", version="1.0.0")

@app.get("/health")
def health_check():
    return {"status": "healthy", "service": "paa-crew"}

@app.post("/analyze", response_model=AnalyzeResponse)
def analyze_portfolio(request: AnalyzeRequest):
    try:
        crew_result = run_pre_analysis_crew(request)
        raw = str(crew_result)
        text = (crew_result.raw if hasattr(crew_result, 'raw')
                else raw)
        j_start, j_end = text.find('{'), text.rfind('}') + 1
        parsed = (json.loads(text[j_start:j_end])
                  if j_start != -1 and j_end > j_start else {})
        kf = parsed.get("key_findings", [])

        return AnalyzeResponse(
            task_id=request.task_id,
            summary=parsed.get("summary", ""),
            risk_level=parsed.get("risk_level", "Medium"),
```

```python
            market_outlook=parsed.get("market_outlook", ""),
            key_findings=(json.dumps(kf, ensure_ascii=False)
                          if isinstance(kf, list) else "[]"),
            raw_content=raw, is_successful=True,
            error_message=None,
            generated_at_utc=datetime.now(
                timezone.utc).isoformat())
    except Exception as e:
        return AnalyzeResponse(
            task_id=request.task_id, summary="",
            risk_level="", market_outlook="",
            key_findings="[]", raw_content="",
            is_successful=False,
            error_message=f"PAA Error: {str(e)}",
            generated_at_utc=datetime.now(
                timezone.utc).isoformat())

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0",
                port=FASTAPI_PORT, reload=True)
```

Listing 4. main.py — FastAPI with /analyze endpoint

3.5 Configuration and Dependencies

```python
import os
from dotenv import load_dotenv
load_dotenv()

def get_llm():
    from crewai import LLM
    return LLM(model=os.getenv("MODEL_NAME", "gpt-4o-mini"),
               temperature=0.3)

FASTAPI_PORT = int(os.getenv("FASTAPI_PORT", "8500"))
```

Listing 5. config.py — LLM configuration

requirements.txt

```
crewai>=0.80.0        crewai-tools>=0.14.0
fastapi>=0.115.0      uvicorn>=0.32.0
pydantic>=2.0.0       python-dotenv>=1.0.0
```

4 .NET Backend Integration

4.1 Service Interface and Implementation

```csharp
// Interface
public interface IPreAnalysisService {
    Task<PreAnalysisReport> GenerateReportAsync(
        int taskId, string userId);
}

// Implementation (abbreviated)
public class PreAnalysisService : IPreAnalysisService {
    private readonly HttpClient _httpClient;
    private readonly IPortfolioRepository _portfolioRepo;
    private readonly IConsultancyTaskRepository _taskRepo;

    public async Task<PreAnalysisReport> GenerateReportAsync(
        int taskId, string userId) {
        var task = await _taskRepo.GetByIdAsync(taskId);
        var portfolios = await _portfolioRepo
            .GetByUserIdWithAssetsAsync(userId); // READ-ONLY

```

```
        var payload = new {
            task_id = task.Id,
            task_title = task.Title,
            task_description = task.Description,
            task_category = task.Category.ToString(),
            user_portfolios = portfolios.Select(p => new {
                portfolio_id = p.Id, name = p.Name,
                assets = p.Assets.Select(a => new {
                    symbol = a.Symbol,
                    asset_type = a.AssetType.ToString(),
                    quantity = a.Quantity,
                    average_cost = a.AverageCost,
                    currency = a.Currency,
                    current_value = a.CurrentValue })
            })
        };

        var resp = await _httpClient
            .PostAsJsonAsync("/analyze", payload);
        resp.EnsureSuccessStatusCode();
        var result = await resp.Content
            .ReadFromJsonAsync<PaaResponse>();

        return new PreAnalysisReport {
            ConsultancyTaskId = taskId,
            Summary = result.Summary ?? "",
            RiskLevel = result.RiskLevel ?? "Medium",
            MarketOutlook = result.MarketOutlook ?? "",
            KeyFindings = result.KeyFindings ?? "[]",
            IsSuccessful = result.IsSuccessful,
            GeneratedAtUtc = DateTime.UtcNow };
    }
}
```

Listing 6. IPreAnalysisService.cs and PreAnalysisService.cs (key excerpt)

4.2 DI Registration and Task Trigger

```
// DI Registration
services.AddHttpClient<IPreAnalysisService,
    PreAnalysisService>(client => {
    client.BaseAddress = new Uri(
        configuration["PaaService:BaseUrl"]
        ?? "http://localhost:8500");
    client.Timeout = TimeSpan.FromSeconds(120);
});

// Task Creation Trigger (inside CreateTaskAsync)
await _taskRepository.AddAsync(task);
try {
    var report = await _preAnalysisService
        .GenerateReportAsync(task.Id, task.UserId);
    await _reportRepository.AddAsync(report);
} catch (Exception ex) {
    _logger.LogWarning(ex,
        "PAA failed for task {Id}", task.Id);
    // Task still created -- graceful degradation
}
```

Listing 7. ServiceRegistration.cs and ConsultancyTaskService.cs

Graceful Degradation

If the PAA service is unavailable, the task is **still created** successfully. The AI layer **never blocks** the core business workflow.

5 Database, Challenges and Security

5.1 Database Schema

The `PreAnalysisReport` entity was pre-designed during HW2 — **no migrations needed**:

Table 3. `PreAnalysisReports` Table

Column	Type	Description
`Id`	`int (PK)`	Auto-increment
`ConsultancyTaskId`	`int (FK, Unique)`	1:1 link to task
`Summary`	`nvarchar(MAX`	AI overview
`RiskLevel`	`nvarchar(50)`	Low / Medium / High
`MarketOutlook`	`nvarchar(MAX`	Observation (not advice)
`KeyFindings`	`nvarchar(MAX`	JSON array
`RawContent`	`nvarchar(MAX`	Full CrewAI output
`IsSuccessful`	`bit`	Pipeline status
`ErrorMessage`	`nvarchar(500`	Truncated error (450 chars)

5.2 Technical Challenges

Table 4. Implementation Challenges and Resolutions

Challenge	Problem	Solution
EF Core 1:1	Delete-reinsert caused Change Tracker conflicts (500).	"Update-if-exists" pattern.
SQL Truncation	Error messages exceeded nvarchar(500).	Auto-truncate to 450 chars.
Python Compat	Python 3.14 incompatible with CrewAI.	Isolated Python 3.12 venv.
JSON Parsing	CrewAI wraps JSON in markdown fences.	Extract first `{` to last `}`.

5.3 Security: Dual-Layer Safety

Table 5. No-Financial-Advice Safety Architecture

Layer	Mechanism	Where
Prompt-Level	Agent backstories: "NEVER give advice", "NEVER use recommend / should / buy / sell".	`agents.py`
Validator	Post-generation regex for prohibited terms (*tavsiye, al/sat, öneriyorum*).	`crew.py`

Additional: PAA has **read-only** DB access; user PII is **stripped** before LLM calls; AI content is **labelled** with disclaimers; API keys stored in **Azure Key Vault**.

6 Working Application Screenshots

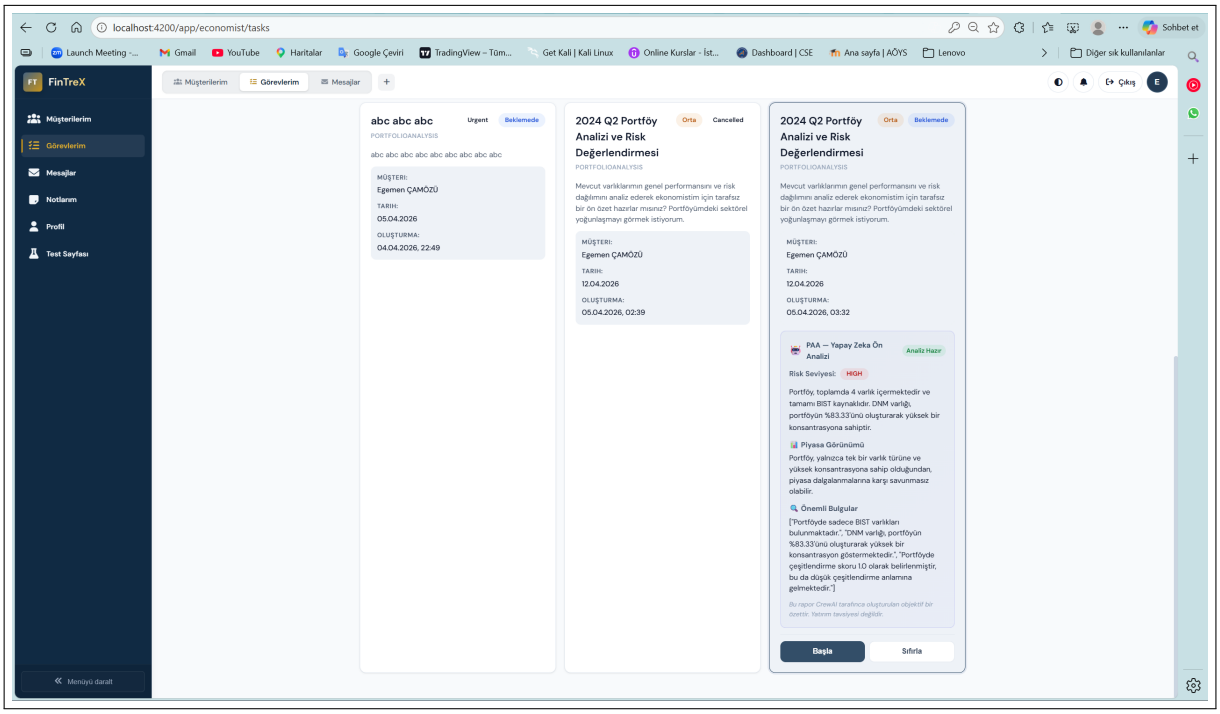

Figure 3. Economist — Görevlerim (My Tasks). Three task cards visible. The rightmost card (teal border) has the **PAA AI pre-analysis card** attached, showing Risk Level: `HIGH`, portfolio summary, market outlook, and key findings — all generated automatically by CrewAI upon task submission.

Figure 4. PAA Card — Detail View

PAA — Yapay Zeka Ön Analizi (AI Pre-Analysis Card)

Close-up of the AI-generated summary card rendered within a consultancy task. Key elements:

1. `Analiz Hazır` badge confirming successful generation.
2. **Risk Level:** `HIGH` — determined by concentration ratio (DNM asset = 83.33% of portfolio).
3. **Piyasa Görünümü** — observes single-type exposure vulnerability.
4. **Önemli Bulgular** — BIST-only allocation, high concentration, diversification score 1.0.
5. **Disclaimer:** "Bu rapor CrewAI tarafınca oluşturulan objektif bir özettir. Yatırım tavsiyesi değildir."

7 End-to-End Flow and Conclusion

7.1 Data Flow Summary

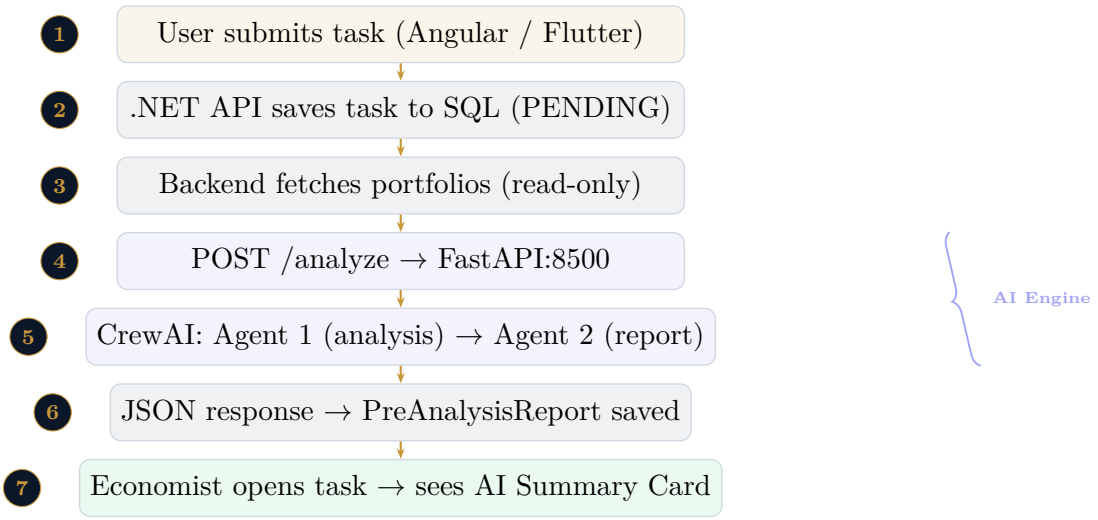

Figure 5. Complete PAA Pipeline

7.2 Future Enhancements

Table 6. Post-MVP Roadmap

Feature	Phase	Description
Async Queue	2	Background jobs via Hangfire
Live Market Tools	2	Yahoo Finance API as CrewAI tool
Report Refresh	2	Economist re-triggers PAA
Multi-LLM	3	OpenAI / Claude / Azure failover
Sentiment Analysis	3	News API for market sentiment

7.3 Conclusion

The CrewAI integration for FinTreX's PAA has been successfully implemented:

1. **Two-agent pipeline** (Data Analyst + Report Writer) with sequential CrewAI orchestration.
2. **FastAPI microservice** bridging .NET and Python with robust error handling.
3. **Seamless .NET integration** via typed HttpClient, DI, and graceful degradation.
4. **Dual-layer safety** ensuring no financial advice is ever generated.
5. **Working frontend** rendering PAA cards with risk level, findings, and disclaimer.

The PAA eliminates the Economist's cold-start problem: when a task arrives, a structured, objective portfolio summary is already waiting — not replacing expertise, but accelerating the workflow.

Advanced Web Programming · Homework 3 · Akdeniz University · 2026 · Egemen ÇAMÖZÜ
FinTreX CrewAI Report — AI-generated content is strictly observational, not financial advice.